Chapter 6 worked solutions

Training and Generating with a Small Transformer

These written solutions cover Exercises 2, 4, and 5. They follow the loss for one prediction and across several positions, calculate the effect of clipping a gradient, and count the tensors stored by a KV cache. Exercises 1, 3, and 6–12 are in the companion notebook.

2. Calculate the signal

At one target position the model has three possible next tokens. Its probability row is $(0.8, 0.15, 0.05)$, and the first class is the reference. The loss at this position uses the reference probability 0.8. Taking its negative natural logarithm gives $-\log(0.8) \approx 0.2231435513$ nats. A lower probability for the reference would give a larger penalty.

Let z_v be the logit for class v, meaning its score before softmax, and let $y = 1$ be the correct class in one-based notation. Write the loss in terms of those scores.

$$\ell(z, y) = -z_y + \log\left(\sum_{j=1}^{3} \exp(z_j)\right).$$

Changing z_v changes the first term by -1 per unit if $v = y$ and by 0 otherwise. In the logarithmic term, only the exponential of z_v changes. The derivative of the logarithm divides by the sum of all exponentials, while the derivative of $\exp(z_v)$ supplies $\exp(z_v)$ in the numerator. Their ratio is the softmax probability p_v. Combining the two terms gives

$$\frac{\partial \ell}{\partial z_v} = \frac{\exp(z_v)}{\sum_j \exp(z_j)} - 1[v = y] = p_v - 1[v = y].$$

Logit	Softmax probability	Reference indicator	Derivative
Class 1	0.80	1	$0.80 - 1 = -0.20$
Class 2	0.15	0	$0.15 - 0 = 0.15$
Class 3	0.05	0	$0.05 - 0 = 0.05$

The negative first derivative says that increasing the correct logit locally lowers the loss. The positive derivatives for the other two classes say that increasing either of their scores locally raises the loss. A gradient-descent step on these logits alone would move in the opposite direction. In the full model, backpropagation uses these derivatives to calculate how shared parameters affect the loss. The entries sum to zero, which agrees with the fact that shifting all three logits by the same constant leaves the probability row unchanged.

The chapter's **three-position** calculation asks a different question. Each of three positions has its own complete vocabulary distribution, and we take the probability of that position's reference token from each row.

Evaluated position	Reference probability	Contribution to the loss sum
1	0.80	$-\log(0.80) \approx 0.2231435513$
2	0.50	$-\log(0.50) \approx 0.6931471806$
3	0.25	$-\log(0.25) \approx 1.3862943611$

There are three evaluated labels. Add their losses before dividing, and keep the unrounded values through the exponential. The three reference probabilities multiply to 0.10, so the log sum also has a simple exact form.

$$\begin{aligned} NLL &= -\frac{\log(0.80) + \log(0.50) + \log(0.25)}{3} = \frac{\log 10}{3} \approx 0.7675283643, \\ perplexity &= \exp(NLL) = 10^{1/3} \approx 2.1544346900. \end{aligned}$$

The first calculation uses three probabilities that sum to one **within a single row**. Its gradient has one entry per vocabulary class. The second uses one selected probability from **each of three rows**. Those selected probabilities sum to 1.55, which is compatible with each complete row summing to one. Perplexity exponentiates the average of the three losses.

4. Limit a gradient

Imagine that two model parameters have gradients 3 and 4. Treat them as one vector $g = (3, 4)$ and cap its Euclidean norm at $c = 1$. The Euclidean norm is the square root of the sum of the squared coordinates. The clipping rule multiplies every coordinate by the same factor $\min(1, c/\|g\|_2)$ for a nonzero gradient.

Original gradient	Original norm	Scale factor	Clipped gradient	New norm
$(3, 4)$	$\sqrt{3^2 + 4^2} = 5$	$1/5 = 0.2$	$(0.6, 0.8)$	$\sqrt{0.36 + 0.64} = 1$
$(0.3, 0.4)$	$\sqrt{0.09 + 0.16} = 0.5$	$\min(1, 1/0.5) = 1$	$(0.3, 0.4)$	0.5

In the first row the vector is five times the desired length, so multiplying by one fifth brings it to the cap. Both coordinates shrink by the same factor. Their ratio stays 3:4, and the resulting vector points in the same direction. In the second row the vector is already within the cap. The rule keeps its length at 0.5 because multiplying by 2 would enlarge a gradient that already satisfies the limit. In a real training step, the implementation uses the combined norm across the parameter gradients, so the same scaling argument applies to the full collection.

Clipping determines the gradient that enters Adam. Adam then updates running estimates of the gradient and of its square. It scales each coordinate using those estimates and multiplies by a learning rate set separately. For example, starting with zero moments, the first bias-corrected Adam update for one nonzero gradient coordinate g_i is

$$\Delta\theta_i = -\eta \frac{g_i}{|g_i| + \varepsilon_{Adam}}.$$

With $g_i = 0.6$ and an Adam denominator constant much smaller than 0.6, the magnitude of this step is approximately η. Clipping a raw gradient coordinate from 3 to 0.6 therefore gives nearly the same first-step magnitude in this example. At later steps, the moments also reflect earlier gradients. The cap of 1 describes the norm of the gradient supplied to the optimizer. The resulting movement of the parameters depends on the learning rate, the moments, and Adam's rescaling.

5. Account for memory

Consider one decoder layer, one batch item, and one attention head. For each consumed target position, self-attention stores a key vector and a value vector. Each has $d_h = 8$ coordinates. The self-KV cache for one such head and batch item therefore holds $2 \cdot 20 \cdot 8 = 320$ elements. Multiplying by four heads, three batch items, and two layers counts the complete set of stored vectors.

Storage	Elements	Bytes at four per element	KiB
Target self-KV	$2 \cdot 2 \cdot 3 \cdot 4 \cdot 20 \cdot 8 = 7,680$	30,720	30
Static source cross-KV	$2 \cdot 2 \cdot 3 \cdot 4 \cdot 6 \cdot 8 = 2,304$	9,216	9
Total	$7,680 + 2,304 = 9,984$	39,936	39

The first factor of 2 counts key and value arrays. The second counts layers. Each decoder layer projects the same encoder memory into its own source keys and values. The source has six stored positions, so the cross-KV row uses $S = 6$ where the self-KV row uses $t = 20$. Divide the byte counts by 1,024 to obtain KiB. Adding another target input would grow the self-KV payload by $2 \cdot 2 \cdot 3 \cdot 4 \cdot 8 \cdot 4 = 1,536$ bytes while leaving the cross-KV payload at 9,216 bytes.

Here t is a count of consumed inputs whose key and value vectors are present, including the starting input if it has been consumed. With zero-based indexing, the twentieth stored input has index 19. Processing that input leaves positions 0 through 19 in the cache, so $t = 20$. A newly selected token becomes a cached input when the decoder consumes it at the following step.

These numbers count the element payload of the two kinds of KV tensors. A full memory measurement would also include model parameters, the final encoder hidden vectors, masks and token IDs, and intermediate query vectors, attention scores, probabilities, and activations. The implementation can also use temporary storage to concatenate a new key or value onto an existing cache. Allocator overhead and temporary buffers can make the peak larger than the 39 KiB of stored elements.